Ex. No.: 9

Experiment Title: Deadlock Avoidance using Banker's Algorithm

Aim:
To find out a safe sequence using Banker's Algorithm for deadlock avoidance.

Algorithm:

1. Initialize `work = available` and `finish[i] = false` for all processes `i`.
2. Find an `i` such that both conditions are true:
    - `finish[i] == false`
    - `Need[i] <= Work`
3. If no such `i` exists, go to step 6.
4. Set `work = work + allocation[i]`
5. Set `finish[i] = true`, go to step 2.
6. If `finish[i] == true` for all `i`, print safe sequence.
7. Else, print that there is no safe sequence.

Program Code

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int n, m, i, j, k;
```

```
// Number of processes and resources
n = 5; // P0, P1, P2, P3, P4
m = 3; // A, B, C

int alloc[5][3] = {
    {0, 1, 0},
    {2, 0, 0},
    {3, 0, 2},
    {2, 1, 1},
    {0, 0, 2}
};

int max[5][3] = {
    {7, 5, 3},
    {3, 2, 2},
    {9, 0, 2},
    {2, 2, 2},
    {4, 3, 3}
};

int avail[3] = {3, 3, 2};

int f[n], ans[n], ind = 0;
for (k = 0; k < n; k++) {
    f[k] = 0;
}

int need[n][m];
for (i = 0; i < n; i++) {
    for (j = 0; j < m; j++)
        need[i][j] = max[i][j] - alloc[i][j];
}

int y = 0;
for (k = 0; k < 5; k++) {
    for (i = 0; i < n; i++) {
        if (f[i] == 0) {
```

```
                int flag = 0;
                for (j = 0; j < m; j++) {
                    if (need[i][j] > avail[j]) {
                        flag = 1;
                        break;
                    }
                }

                if (flag == 0) {
                    ans[ind++] = i;
                    for (y = 0; y < m; y++)
                        avail[y] += alloc[i][y];
                    f[i] = 1;
                }
            }
        }
    }

    int safe = 1;
    for (i = 0; i < n; i++) {
        if (f[i] == 0) {
            safe = 0;
            break;
        }
    }

    if (safe) {
        printf("The SAFE Sequence is: ");
        for (i = 0; i < n - 1; i++)
            printf("P%d -> ", ans[i]);
        printf("P%d\n", ans[n - 1]);
    } else {
        printf("System is not in a safe state.\n");
    }

    return 0;
}
```

Sample Output:

```
The SAFE Sequence is: P1 -> P3 -> P4 -> P0 -> P2
```

Result:

The program successfully finds a safe sequence using Banker's algorithm, ensuring deadlock avoidance.